

PROGRAMACIÓN ORIENTADA A OBJETOS – POO104

Taller en Clase 2 (Acumulado) – 20 %

Desafío de programación: OPERACIÓN ORBITA – Centro de Control de Misiones Científicas

Modalidad: Equipos de hasta 3 estudiantes

Tiempo aproximado de resolución: 4 horas

Ponderación: 20 % de la asignatura

Lenguaje: C#

Entorno de desarrollo: Visual Studio

Persistencia: Base de datos mediante el framework de conexión estudiado en clase

1. Contexto del desafío

La organización ficticia **ORBITA – Organización Regional de Búsqueda, Investigación y Tecnología Avanzada** administra diferentes misiones científicas y de exploración.

Para ejecutar cada misión se dispone de distintos recursos tecnológicos, entre ellos vehículos terrestres, drones y estaciones de sensores. Cada recurso posee características particulares, costos de operación, restricciones de utilización y estados de disponibilidad.

Actualmente, la información se administra mediante registros independientes, lo cual ha provocado asignaciones duplicadas, utilización de equipos en mantenimiento, pérdida de información y ausencia de control sobre las operaciones realizadas.

Se requiere desarrollar un prototipo funcional denominado **ORBITA Control**, capaz de gestionar usuarios, recursos y misiones utilizando los principios de Programación Orientada a Objetos estudiados durante el ciclo.

El sistema deberá impedir operaciones inválidas y conservar la información principal en una base de datos.

2. Objetivo general

Desarrollar una solución en C# que permita administrar misiones científicas y los recursos asignados a ellas, aplicando modelado de objetos, encapsulamiento, herencia, polimorfismo, abstracción, interfaces, colecciones, manejo de excepciones, persistencia de datos, sesiones de usuario y separación del sistema en diferentes proyectos o capas.

3. Situación problemática

Cada misión deberá poseer como mínimo:

- Código único.
- Nombre.
- Descripción.
- Fecha de inicio.
- Fecha estimada de finalización.

- Nivel de prioridad.
- Estado.
- Responsable.
- Recursos asignados.

Se deberán manejar los siguientes estados:

Planificada – En ejecución – Finalizada – Cancelada

Los recursos disponibles para las misiones podrán ser de diferentes tipos.

Dron

Deberá registrar, como mínimo:

- Código.
- Modelo.
- Autonomía de vuelo.
- Alcance.
- Costo por hora.
- Estado.

Rover terrestre

Deberá registrar:

- Código.
- Modelo.
- Autonomía.
- Capacidad de carga.
- Costo por kilómetro.
- Estado.

Estación de sensores

Deberá registrar:

- Código.
- Modelo.
- Cantidad de sensores.
- Consumo energético.

- Costo diario.
- Estado.

Para todos los recursos deberán considerarse al menos los estados:

Disponible – Asignado – Mantenimiento

4. Requerimientos de Programación Orientada a Objetos

4.1. Modelado mediante clases y objetos

Deberá diseñarse un modelo de objetos coherente con el problema.

Como mínimo deberán identificarse las entidades:

- Usuario.
- Misión.
- Recurso de exploración.
- Asignación de recurso.

Podrán incorporarse otras clases cuando el diseño propuesto lo requiera.

Antes de iniciar la implementación deberá elaborarse un **diagrama de clases UML simplificado** que muestre:

- Clases principales.
- Atributos relevantes.
- Métodos principales.
- Relaciones.
- Herencia.
- Interfaces implementadas.

4.2. Clase abstracta

Los diferentes recursos deberán derivarse de una clase abstracta denominada, por ejemplo:

RecursoExploracion

La clase deberá contener los atributos y comportamientos comunes.

Se deberá incluir como mínimo un método abstracto relacionado con el funcionamiento del recurso, por ejemplo:

CalcularCostoOperacion(...)

Cada tipo de recurso deberá implementar su propia forma de realizar dicho cálculo.

El diseño deberá demostrar el uso apropiado de:

- Herencia.
- Clase abstracta.
- Sobrescritura de métodos.
- Polimorfismo.

4.3. Constructores y sobrecarga

Deberán utilizarse constructores para garantizar que los objetos sean creados en estados válidos.

Se deberá implementar al menos un caso de **sobrecarga de constructores o métodos**.

Por ejemplo, un recurso podrá crearse:

- Con todos sus datos.
- Con únicamente los datos mínimos necesarios.

No se deberán utilizar objetos cuyos atributos esenciales queden en estados inconsistentes.

4.4. Encapsulamiento y modificadores de acceso

Se deberá aplicar correctamente:

- public
- private
- protected

cuando el diseño lo requiera.

No se considerará apropiado declarar todos los atributos como públicos únicamente para facilitar su utilización.

5. Interfaces

Deberá definirse e implementarse al menos **una interfaz propia**.

Se sugiere utilizar una interfaz semejante a:

IAsignable

con operaciones como:

- Asignar a una misión.
- Liberar recurso.
- Consultar disponibilidad.

También podrá proponerse una interfaz diferente siempre que su utilización se encuentre debidamente justificada.

La interfaz deberá ser implementada por dos o más clases cuando la lógica del modelo lo permita.

6. Colecciones y estructuras dinámicas

Durante la ejecución del programa deberá utilizarse al menos una colección genérica de C#, por ejemplo:

List<T>

La colección podrá utilizarse para:

- Recursos de una misión.
- Resultados de consultas.
- Historial temporal de operaciones.
- Recursos disponibles.
- Misiones activas.

Se deberá demostrar el recorrido, búsqueda, inserción o eliminación de elementos de la colección.

7. Polimorfismo: cálculo de operación

Cada recurso deberá calcular su costo operativo de forma diferente.

Por ejemplo:

Dron

Costo = horas de vuelo × costo por hora

Rover

Costo = kilómetros recorridos × costo por kilómetro

Estación de sensores

Costo = días de utilización × costo diario

El cálculo deberá realizarse mediante un comportamiento polimórfico definido desde la clase base.

Posteriormente deberá ser posible recorrer una colección de recursos y calcular el costo total estimado de una misión sin comprobar manualmente el tipo concreto de cada objeto.

No se considerará adecuada una solución basada exclusivamente en cadenas extensas de instrucciones if o switch para determinar el tipo de recurso.

8. Manejo de System.Object

Se deberá sobrescribir al menos uno de los métodos heredados de System.Object.

Se recomienda:

ToString()

La implementación deberá generar una representación comprensible del objeto.

Ejemplo conceptual:

DRN-004 | Dron Falcon X2 | Disponible

9. Manejo de excepciones

El sistema deberá controlar adecuadamente las situaciones excepcionales.

Como mínimo deberán validarse situaciones como:

- Código duplicado.
- Campos obligatorios vacíos.
- Valores numéricos negativos.
- Fechas inválidas.
- Intento de asignar un recurso que no se encuentre disponible.
- Intento de iniciar una misión sin recursos.
- Credenciales incorrectas.
- Errores producidos durante el acceso a datos.

Se deberá utilizar:

- try
- catch
- finally

cuando resulte pertinente.

Además, deberá implementarse al menos **una excepción personalizada**, por ejemplo:

RecursoNoDisponibleException

o una equivalente propuesta por el equipo.

No se considerará suficiente utilizar bloques try-catch vacíos o utilizar excepciones como sustituto de toda validación.

10. Persistencia en base de datos

La información principal deberá almacenarse utilizando una base de datos y el framework de conexión estudiado durante la asignatura.

Como mínimo deberán persistirse:

Usuarios

- Id.

- Nombre de usuario.
- Contraseña o mecanismo equivalente utilizado durante el taller.
- Rol.
- Estado.

Misiones

- Id.
- Código.
- Nombre.
- Descripción.
- Prioridad.
- Fecha de inicio.
- Fecha de finalización.
- Estado.

Recursos

- Id.
- Código.
- Nombre o modelo.
- Tipo.
- Estado.
- Información necesaria para su operación.

Asignaciones

Deberá establecerse una relación que permita conocer qué recursos pertenecen o han sido asignados a cada misión.

11. Operaciones mínimas

El programa deberá permitir realizar como mínimo las siguientes operaciones:

Usuarios

- Iniciar sesión.
- Identificar al usuario autenticado.
- Cerrar sesión.

Recursos

- Registrar un recurso.
- Modificar sus datos.
- Consultar recursos.
- Consultar únicamente los recursos disponibles.
- Cambiar el estado de un recurso.

Misiones

- Crear una misión.
- Modificar una misión.
- Consultar las misiones registradas.
- Asignar recursos.
- Retirar recursos.
- Iniciar una misión.
- Finalizar una misión.
- Calcular el costo estimado de operación.

12. Sesiones y roles de usuario

Se deberán manejar al menos tres tipos de usuario:

Administrador

Podrá administrar usuarios, recursos y misiones.

Coordinador

Podrá:

- Crear y modificar misiones.
- Asignar recursos.
- Iniciar o finalizar misiones.
- Consultar información.

Auditor

Dispondrá únicamente de operaciones de consulta.

El sistema deberá mantener la información del usuario que ha iniciado sesión y utilizarla para controlar el acceso a los diferentes módulos.

No será válido limitarse a mostrar opciones distintas en pantalla si las operaciones restringidas pueden ejecutarse directamente sin ninguna comprobación.

13. Regla especial: protocolo de seguridad ORBITA

Antes de iniciar una misión se deberán comprobar automáticamente las siguientes condiciones:

1. La misión deberá contener al menos un recurso.
2. Ningún recurso podrá encontrarse en mantenimiento.
3. Ningún recurso podrá estar asignado a otra misión activa.
4. La fecha de finalización estimada no podrá ser anterior a la fecha de inicio.
5. El responsable de la misión deberá encontrarse activo.
6. Todos los datos obligatorios deberán encontrarse completos.

Si alguna condición no se cumple, la misión no podrá iniciar y deberá mostrarse una explicación comprensible.

14. Consulta especial: Panel de Control ORBITA

Deberá desarrollarse una opción que presente como mínimo:

- Cantidad total de misiones registradas.
- Misiones planificadas.
- Misiones actualmente en ejecución.
- Misiones finalizadas.
- Cantidad de recursos disponibles.
- Cantidad de recursos asignados.
- Cantidad de recursos en mantenimiento.
- Costo estimado total de una misión seleccionada.

Los datos deberán obtenerse a partir de los objetos y/o registros almacenados y no mediante valores escritos directamente en el código.

15. Organización de la solución

La solución deberá estar conformada por **al menos tres proyectos o componentes claramente diferenciados**.

Una posible organización será:

ORBITA.Dominio

Contendrá:

- Clases.
- Clases abstractas.
- Interfaces.
- Enumeraciones.
- Excepciones propias.
- Reglas principales del dominio.

ORBITA.Datos

Contendrá:

- Contexto de base de datos.
- Configuración de conexión.
- Operaciones relacionadas con persistencia.

ORBITA.App

Contendrá:

- Menús o interfaz de interacción.
- Inicio y cierre de sesión.
- Acceso a los diferentes módulos.

Podrá utilizarse otra estructura siempre que exista una separación lógica y justificable de responsabilidades.

16. Principios SOLID

En el diseño realizado deberán identificarse **al menos dos principios SOLID** que se encuentren reflejados en la solución.

Se deberá explicar brevemente:

- Qué principio se está aplicando.
- En qué parte del código puede identificarse.
- Qué problema se pretende evitar mediante su utilización.

No será suficiente mencionar únicamente el nombre del principio.

17. Patrones de diseño

Debido a que los patrones de diseño se encuentran en etapa introductoria, no será obligatorio desarrollar una implementación avanzada.

Sin embargo, deberá identificarse **un patrón de diseño que podría resultar conveniente para una versión futura de ORBITA Control.**

Se deberá indicar:

- Nombre del patrón.
- Clasificación: creacional, estructural o de comportamiento.
- Problema del sistema que podría resolver.
- Clases que podrían participar.

La propuesta podrá representarse mediante un pequeño diagrama UML o una explicación técnica breve.

La implementación del patrón se considerará opcional siempre que no interfiera con los demás requerimientos.

18. Restricciones del desafío

No se permitirá:

- Resolver todo el sistema dentro de una sola clase.
- Utilizar únicamente variables globales o métodos estáticos para representar el modelo.
- Sustituir la herencia mediante múltiples variables que indiquen el tipo del objeto.
- Eliminar el control de excepciones.
- Mantener toda la información únicamente en memoria cuando se exige persistencia.
- Crear opciones de menú sin implementar su funcionalidad.
- Utilizar código perteneciente a otro equipo.
- Incorporar funcionalidades que impidan ejecutar o evaluar las características obligatorias.

No será necesario desarrollar una interfaz gráfica compleja. La aplicación podrá funcionar mediante consola o mediante la tecnología de interfaz utilizada previamente durante la asignatura.

19. Datos de prueba obligatorios

Antes de la evaluación deberán existir como mínimo:

- 3 usuarios con roles diferentes.
- 3 drones.
- 3 rovers.
- 3 estaciones de sensores.
- 4 misiones.

- Una misión en ejecución.
- Una misión finalizada.
- Una misión planificada.
- Al menos un recurso en mantenimiento.

Deberá demostrarse también un intento fallido de asignación o ejecución para comprobar el manejo de reglas y excepciones.

20. Entregables

Al finalizar las ocho horas deberá entregarse:

1. **Solución completa de Visual Studio.**
2. **Base de datos o script necesario para su creación**, según la tecnología utilizada.
3. **Diagrama de clases UML.**
4. **Código fuente ejecutable. (todo el código de su aplicación puede ser en un .txt**
5. **Archivo README** con:
 - Integrantes.
 - Instrucciones de ejecución.
 - Credenciales de prueba.
 - Distribución general de módulos.
 - Principios SOLID identificados.
 - Patrón de diseño propuesto.
6. Evidencia de las pruebas realizadas.

El proyecto deberá compilar y ejecutarse en el momento de la revisión.

21. Evaluación

La calificación del desafío se realizará sobre **100 puntos**, equivalentes al **20 % de la nota final de la asignatura**.

Criterio	Puntos
Modelado de clases y calidad del diagrama UML	10
Clases, objetos, constructores, sobrecarga y encapsulamiento	10
Herencia, clase abstracta y jerarquía de clases	10

Criterio	Puntos
Polimorfismo e interfaces	10
Colecciones y manejo de objetos	5
Manejo de excepciones y validaciones	10
Persistencia y conexión con base de datos	15
Sesiones, roles y módulos del sistema	10
Organización multiproyecto y separación de responsabilidades	5
Aplicación y explicación de principios SOLID	5
Funcionalidad integral y cumplimiento de reglas del negocio	5
Calidad del código, documentación y pruebas	5
Total	100

22. Penalizaciones

Podrán aplicarse las siguientes deducciones:

- Proyecto que no compile: hasta **-20 puntos**.
- Credenciales o configuración necesarias no proporcionadas: hasta **-5 puntos**.
- Funcionalidades declaradas pero no implementadas: según el criterio correspondiente.
- Base de datos no disponible o imposible de restaurar/crear: hasta **-15 puntos**.
- Ausencia del diagrama UML: **-10 puntos**.
- Uso injustificado de código duplicado o solución completamente monolítica: hasta **-10 puntos**.

Una misma deficiencia no deberá penalizarse de forma duplicada en diferentes criterios.

23. Distribución sugerida de las 4 horas

La administración del tiempo será responsabilidad del equipo. Como referencia, se recomienda la siguiente distribución:

Etapas	Tiempo sugerido
Análisis del problema y UML	22 min
Creación de solución y modelo de dominio	37 min

Etapas	Tiempo sugerido
Base de datos y persistencia	45 min
Sesiones, usuarios y módulos	30 min
Misiones, recursos y reglas de negocio	15 min
Integración, consultas y panel de control	23 min
Pruebas, correcciones y documentación	37 min
Total	4 horas

24. Condición final del desafío

No se evaluará únicamente la cantidad de opciones implementadas.

Se valorará principalmente que la solución demuestre que los diferentes conceptos estudiados pueden integrarse en un sistema coherente.

Una solución pequeña, correctamente modelada, persistente, validada y estructurada tendrá mayor valoración que una aplicación con numerosas pantallas o menús cuyo código no aplique adecuadamente Programación Orientada a Objetos.

Desafío adicional opcional

Cuando todos los requerimientos obligatorios hayan sido completados, podrá incorporarse una **bitácora de actividad** que registre:

- Usuario.
- Fecha y hora.
- Acción realizada.
- Entidad afectada.
- Descripción.

La bitácora podrá utilizarse para demostrar consultas adicionales, separación de responsabilidades y propuestas relacionadas con patrones de diseño.